Guia de Engenharia DevSecOps Nível 3 (Técnica)

Projeto Final DevSecOps Clube Aurora (Turma 1746)

> Bem-vindo(a) ao **Nível 3 (Técnica)**!
> Este guia é o seu manual de referência avançado para a remediação integral das 7 falhas de segurança do Clube Aurora (**F1 a F7**). Aqui você atuará como Engenheiro(a) de DevSecOps sênior, depurando problemas localmente, entendendo o encadeamento dos gates de segurança da pipeline de CI/CD e configurando exceções auditáveis.

---

1. Visão Geral e Estratégia de Execução

1.1 O Desafio Técnico Completo

No Nível 3, sua meta não é apenas "passar nos testes", mas implementar **segurança defensiva em profundidade**:
- **Escopo 100% Prático:** Você aplicará correções em arquivos Python (`app/db.py` , `app/main.py`), arquivos de configuração de contêineres (`Dockerfile` , `docker-compose.yml`), manifestos de dependências (`requirements.txt`) e higiene de repositório (`.gitignore` , `.env.example`).
- **Pipeline 100% Verde:** Ao submeter seu Pull Request, tanto a **Pipeline Estática** (SAST, SCA e Linter) quanto a **Pipeline Dinâmica** (Container Security e DAST) devem concluir com sucesso (ícones verdes).
- **Entregáveis:** Repositório fork saneado, Pull Request aberto para `main` e o relatório `aluno/TEMPLATE_EVIDENCIAS.md` preenchido com evidências reais do antes e do depois.

1.2 Ordem Estratégica de Resolução (Por que a ordem importa)

Na engenharia de software, tentar corrigir tudo ao mesmo tempo gera confusão e ciclos lentos de feedback. Recomendamos fortemente a seguinte ordem sequencial de atuação:

```
[1. F2: SCA Deps]  [2. F3: Semgrep SQLi]  [3. F4: Dockerfile Hardening]

                          (Pipeline Estática 100% Verde + Trivy Image Verde!)
[4. F1: Segredos Git]  [5. F7: Compose IaC]  [6. F6: RBAC Autorização]  [7. F5: Headers HTTP ZAP]
```

Por que essa sequência é a mais inteligente?
1. **F2 e F3 limpam a Pipeline Estática:** Semgrep e Trivy FS deixam de falhar imediatamente.
2. **F4 vira a Pipeline Dinâmica:** O contêiner antigo continha 14 vulnerabilidades que abortavam o job no step `Trivy Container Image Scan` . Ao endurecer o `Dockerfile` com imagem base segura e pacotes atualizados, o Trivy Image zera as falhas.
3. **F5 passa a ser testado pelo ZAP:** Com a imagem segura, a aplicação Flask finalmente consegue inicializar em background (`Inicializar Aplicação`), permitindo que o scanner OWASP ZAP execute sua análise dinâmica.

1.3 Mapeamento de Dependências da Pipeline (Fail-Fast e o bloqueio do ZAP)

O workflow `.github/workflows/devsecops.yml` opera no modelo **Fail-Fast** por padrão. Isso significa que, se um step falhar com código de saída diferente de zero (`exit code 1`), todos os steps subsequentes daquele

job que não possuam a diretiva `if: always()` são **cancelados sumariamente**.

Na versão vulnerável original:
- O step `Trivy Container Image Scan` falha com código `1`.
- O step `Inicializar Aplicação` é ignorado.
- O step `OWASP ZAP Baseline Scan` sequer chega a rodar.

Portanto, **a correção de F2 e F4 é pré-requisito técnico absoluto** para que você consiga validar a falha F5 dinamicamente no GitHub Actions.

---

2. Como Investigar os Achados

2.1 Lendo as Tabelas do Trivy e Logs do Hadolint

- **Trivy FS (SCA):** Examine o log do step `Análise de Dependências (Trivy FS)`. A tabela lista cada pacote instalado, a severidade do achado e a coluna fundamental: `Fixed Version`.
- **Hadolint (Linter):** No step `Análise de Dockerfile (Hadolint)`, procure por regras iniciadas em `DL` (ex: `DL3007` para tag `:latest`, `DL3002` para troca de usuário, etc.).
- **Trivy Image:** Inspeciona a imagem compilada, verificando tanto as bibliotecas Python instaladas quanto os pacotes do sistema operacional da base Debian/Alpine.

2.2 Inspecionando Arquivos SARIF na aba Security

O pipeline exporta os resultados do Trivy e do Semgrep no formato padronizado **SARIF (Static Analysis Results Interchange Format)** e faz o upload para o GitHub:
1. No seu repositório, clique na aba **Security**.
2. No menu lateral esquerdo, clique em **Code scanning**.
3. Você verá todos os alertas abertos correlacionados diretamente com as linhas do código-fonte, com descrições das regras e referências do CWE/OWASP.

2.3 Baixando e Analisando Artefatos do Workflow

Ao final de cada execução da pipeline, na parte inferior da página do workflow run (seção **Artifacts**), você pode baixar os seguintes relatórios em formato bruto:
- `trivy-fs-report` (trivy-fs-results.json / SARIF)
- `hadolint-report` (hadolint-results.json)
- `semgrep-report` (semgrep-results.json / SARIF)
- `trivy-image-report` (trivy-image-results.json / SARIF)
- `zap-report` (zap-report.html / zap-report.md)

2.4 Como Rodar os Scanners Localmente com Docker (Depuração Rápida)

Para não depender do tempo de fila do GitHub Actions a cada pequeno ajuste, você pode rodar os mesmos scanners diretamente no terminal do seu computador (requer Docker instalado):

Rodando o Trivy FS localmente:

```
docker run --rm -v "${PWD}:/src" aquasec/trivy:0.51.2 fs --severity HIGH,CRITICAL --scanners vuln /src
```

Rodando o Semgrep localmente:

```
docker run --rm -v "${PWD}:/src" semgrep/semgrep:1.78.0 semgrep --config auto /src/app
```

Rodando o Hadolint localmente:

```
# No Linux / macOS / Git Bash:
docker run --rm -i hadolint/hadolint:v2.12.0 < Dockerfile

# No PowerShell (Windows):
Get-Content Dockerfile | docker run --rm -i hadolint/hadolint:v2.12.0
```

Construindo e testando a imagem localmente:

```
docker build -t aurora-app:test .
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:0.51.2 image --severity HIGH,CR
```

2.5 Como Rodar a Autoavaliação Local (Self-Check sem Docker)

Se você não tiver Docker configurado ou quiser uma validação instantânea de todas as 7 falhas em milissegundos usando apenas o Python local:

```
python scripts/selfcheck.py --nivel 3
```

- O script audita localmente as regras de F1 a F7 e retorna `exit code 0` quando todas as falhas estiverem sanadas.
- Caso alguma falha persista, o script exibe dicas objetivas e pontuais indicando exatamente o que falta ajustar.

---

3. Resolução Cirúrgica das 7 Falhas (F1 a F7)

F2 Dependências Python Vulneráveis (`requirements.txt`)

Sintoma

No step `Análise de Dependências (Trivy FS)` , o scanner acusa 10 vulnerabilidades de severidade `HIGH` nos pacotes `gunicorn (20.1.0)` , `urllib3 (1.26.5)` e `werkzeug (2.2.3)` (CVE-2024-1135, CVE-2024-6827, CVE-2023-43804, CVE-2024-34069, entre outras).

Hipótese Técnica

Bibliotecas antigas possuem falhas conhecidas de HTTP request smuggling, vazamento de cookies em redirecionamentos e vulnerabilidades de ReDoS no parser de rotas.

Como Testar Localmente

```
docker run --rm -v "${PWD}:/src" aquasec/trivy:0.51.2 fs --severity HIGH,CRITICAL --scanners vuln /src
```

[NOTA] Pistas Progressivas (F2) - **Pista 1:** A coluna `Fixed Version` do log indica que o `gunicorn` deve ir para `>=23.0.0` e o `werkzeug` para `>=3.0.3`. - **Pista 2:** Como o `flask` original (`2.2.5`) é rigidamente amarrado a versões antigas do `werkzeug`, atualize também o `flask` para `3.0.3` e o `urllib3` para `>=2.2.2`

(ou `2.8.0`).

Diretrizes de Implementação

- **Atualização Semântica:** Inspecione a coluna `Fixed Version` da tabela do Trivy e declare as versões recomendadas no `requirements.txt`.
- **Atenção ao Acoplamento:** Atualize o `werkzeug` em conjunto com o `flask` para manter total consistência entre os pacotes do ecossistema Pallets.
- **Roteiro de Depuração CLI:** Para verificar sintomas de quebra de import ou rodar o teste local de SCA, consulte `aluno/PLANO_DE_TESTES_DEBUGGING.md`.

Prova de Validação

O comando de scan do Trivy FS retorna:

```
Total: 0 (HIGH: 0, CRITICAL: 0)
```

F3 Injeção de SQL via Interpolação de Strings (`app/db.py`)

Sintoma

No step `Análise SAST (Semgrep)`, duas regras falham na linha 102 de `app/db.py`:
- `python.lang.security.audit.formatted-sql-query`
- `python.sqlalchemy.security.audit.sqlalchemy-execute-raw-query`

Hipótese Técnica

A função `search_clients(name)` monta a consulta SQL através de interpolação direta (`f"SELECT ... LIKE '%{name}%'"`). Se o argumento `name` contiver aspas simples ou metacaracteres SQL, o invasor altera a estrutura sintática da query no SQLite.

Como Testar Localmente

```
docker run --rm -v "${PWD}:/src" semgrep/semgrep:1.78.0 semgrep --config auto /src/app
```

[NOTA] Pistas Progressivas (F3) - **Pista 1:** A interpolação de strings deve ser completamente eliminada da instrução SQL. - **Pista 2:** No SQLite / Python DB-API, o caractere de substituição parametrizada é o ponto de interrogação `?`. O wildcard `%` deve fazer parte do valor do parâmetro passado na tupla, e não estar hardcoded na string SQL.

Diretrizes de Implementação

- **Eliminação de f-string:** Remova qualquer interpolação direta de texto na instrução SQL.
- **Parâmetros DB-API:** Utilize o placeholder `?` nativo do SQLite na string da query e passe os valores com os wildcards `%` na tupla de parâmetros do método `.execute()`.
- **Roteiro de Depuração CLI:** Para testar a regra do Semgrep e validar a suíte de testes unitários localmente, consulte `aluno/PLANO_DE_TESTES_DEBUGGING.md`.

Prova de Validação

O Semgrep relata `0 findings` e o exit code é `0`.

F4 Hardening do Contêiner (`Dockerfile`)

Sintoma

- **Hadolint:** Alerta `DL3007` na linha 6 por usar `FROM python:latest`.
- **Trivy Image:** Alerta 14 vulnerabilidades críticas/altas nos pacotes Debian da imagem base.
- **Auditoria de Hardening:** O contêiner roda como `root`, não tem `HEALTHCHECK` e inclui ferramentas desnecessárias em tempo de execução.

Hipótese Técnica

Imagens base `:latest` são mutáveis e pesadas. Executar o processo como `root` permite que qualquer exploração de vulnerabilidade na aplicação resulte em controle administrativo do contêiner.

Como Testar Localmente

```
Get-Content Dockerfile | docker run --rm -i hadolint/hadolint:v2.12.0
```

[NOTA] Pistas Progressivas (F4) - **Pista 1:** Use uma imagem minimalista e com versão fixada: `python:3.11-slim`. - **Pista 2:** Implemente **Multi-stage Build**: compile e instale as dependências com `pip` em um primeiro estágio (`AS builder`) e copie apenas o resultado compilado para a imagem final de runtime. - **Pista 3:** No estágio final, atualize pacotes da base Debian (`apt-get update && apt-get upgrade -y`), crie um usuário sem privilégios (`useradd -u 10001 -m nonroot`), declare `USER nonroot` e defina um `HEALTHCHECK` apontando para `/health`.

Diretrizes de Implementação

- **Multi-Stage Build:** Estruture o Dockerfile com um estágio `builder` dedicado para compilação das dependências e um estágio final de runtime imutável e enxuto.
- **Imagem Base e Atualizações:** Adote imagem mínima estável (ex: `python:3.11-slim`) e aplique as atualizações de segurança do sistema operacional da base Debian.
- **Usuário sem Privilégios:** Crie um usuário dedicado (ex: `nonroot` UID 10001), ajuste as permissões de gravação da pasta `/app/instance` para o SQLite e ative `USER nonroot`.
- **Monitoramento:** Declare a instrução `HEALTHCHECK` consultando o endpoint `/health`.
- **Roteiro de Depuração CLI:** Para comandos de compilação, linter Hadolint e resolução de erros de permissão de escrita no SQLite, consulte `aluno/PLANO_DE_TESTES_DEBUGGING.md`.

Prova de Validação

- Hadolint passa sem erros.
- Trivy Image scan conclui com:
  `Total: 0 (HIGH: 0, CRITICAL: 0)`

F1 Higiene de Segredos (`.env` e `.gitignore`)

Sintoma

Arquivo `.env` versionado no Git com credenciais sensíveis e ausente no `.gitignore`.

Hipótese Técnica

Segredos no repositório vazam para clones locais, forks e pipelines, permitindo que qualquer pessoa com acesso de leitura descriptografe sessões ou assuma contas de administrador.

Como Testar Localmente

```
git ls-files .env
```

[NOTA] Pistas Progressivas (F1) - **Pista 1:** O `.gitignore` precisa listar expressamente o arquivo `.env`. - **Pista 2:** Adicionar ao `.gitignore` não remove arquivos que já estavam rastreados pelo Git. Você deve executar `git rm --cached .env`. - **Pista 3:** Disponibilize um `.env.example` para que novos desenvolvedores saibam quais chaves configurar.

Diretrizes de Implementação

- **Regras de Exclusão:** Adicione `.env` ao `.gitignore` para bloquear versionamentos futuros.
- **Modelo de Configuração:** Forneça um `.env.example` com chaves estruturadas e valores genéricos de placeholder.
- **Desindexação no Git:** Remova o arquivo sensível do índice do Git (`git rm --cached .env`) sem apagá-lo do seu ambiente local de desenvolvimento.
- **Roteiro de Depuração CLI:** Para validar o estado do repositório e checagens de segredos, consulte `aluno/PLANO_DE_TESTES_DEBUGGING.md`.

Prova de Validação

`git ls-files .env` retorna vazio e `git status` mostra o repositório limpo.

F7 Hardening de Infraestrutura como Código (`docker-compose.yml`)

Sintoma

No arquivo `docker-compose.yml`, encontramos flags de alto risco: `privileged: true`, montagem de `/var/run/docker.sock`, porta de desenvolvimento `5000:5000` e chave secreta inline.

Hipótese Técnica

A flag `privileged: true` e a exposição do socket do Docker no contêiner anulam qualquer isolamento do kernel Linux, viabilizando **Container Breakout** (escape de contêiner e takeover completo do host).

[NOTA] Pistas Progressivas (F7) - **Pista 1:** Remova `privileged: true` e a seção de volumes com `docker.sock`. - **Pista 2:** Ajuste a porta mapeada para a porta padrão de produção (`8080:8080`) e utilize referências de variáveis de ambiente no `SECRET_KEY`.

Diretrizes de Implementação

- **Menor Privilégio no Daemon:** Remova imediatamente `privileged: true` e a montagem direta do socket `/var/run/docker.sock` do contêiner da aplicação.
- **Higiene de Portas e Segredos:** Mapeie exclusivamente a porta de produção (`8080:8080`) e utilize referências a variáveis de ambiente em vez de senhas hardcoded.
- **Isolamento de Volumes:** Utilize volumes específicos para persistência de dados em vez de montar o sistema de arquivos raiz do hospedeiro.
- **Roteiro de Depuração CLI:** Para validar a sintaxe da orquestração e testes de inicialização local, consulte `aluno/PLANO_DE_TESTES_DEBUGGING.md`.

Prova de Validação

Inspecione o arquivo corrigido: sem `privileged`, sem `docker.sock` montado e sem portas de desenvolvimento abertas.

F6 Quebra de Privilégio Mínimo e RBAC (`app/db.py` e `app/main.py`)

Sintoma

1. Em `app/db.py` (linha 58): `create_user` cria usuários com papel `admin` por padrão (`role="admin"`).
2. Em `app/main.py` (linhas 92 a 94): a rota `/admin/users/<int:user_id>/role` possui apenas `@auth.login_required` , permitindo que qualquer usuário autenticado altere papéis.

Hipótese Técnica

Violação de controle de acesso (Broken Access Control). Usuários comuns conseguem se auto-promover para administradores (elevação vertical de privilégios).

[NOTA] Pistas Progressivas (F6) - **Pista 1:** O papel default ao registrar um usuário deve ser `"client"`. - **Pista 2:** O decorador `@auth.require_role("admin")` já existe na aplicação. Ele deve ser adicionado na rota `/admin/users//role`. - **Atenção à ordem dos decoradores em Flask:** O decorador de rota `@bp.post(...)` deve ficar **por fora** (acima), e o decorador de verificação `@auth.require_role("admin")` logo abaixo dele, imediatamente antes da função `def admin_set_role(user_id):`.

Diretrizes de Implementação

- **Perfil Default Seguro:** Altere o valor default do parâmetro de criação de usuários em `app/db.py` para `"client"` .
- **Proteção de Rota com Decorador:** Em `app/main.py` , proteja a rota `/admin/users/<int:user_id>/role` aplicando o decorador `@auth.require_role("admin")` logo abaixo da declaração de rota do Flask.
- **Garantia de Não-Regressão:** Execute a suíte de testes unitários para assegurar que o seed de bootstrap e os fluxos de login continuam funcionando.
- **Roteiro de Depuração CLI:** Para detalhes sobre RBAC e execução de testes automatizados, consulte `aluno/PLANO_DE_TESTES_DEBUGGING.md` .

Prova de Validação

Execute o teste automatizado ou o validador do curso:

```
python professor/validate.py aurora
# ou
powershell -ExecutionPolicy Bypass -File professor/validate.ps1 aurora
```

O check de F6 passará com sucesso!

F5 Defesa em Profundidade e Cabeçalhos HTTP (`app/main.py` e `app/__init__.py`)

Sintoma

O OWASP ZAP Baseline relata ausência de `X-Content-Type-Options` , `X-Frame-Options` , `Content-Security-Policy` e `Strict-Transport-Security` .

Hipótese Técnica

A aplicação não declara cabeçalhos defensivos para o navegador, deixando clientes vulneráveis a Clickjacking, ataques de MIME-type sniffing e Cross-Site Scripting (XSS).

Como Testar Localmente

Inicie a aplicação e faça uma requisição inspecionando os cabeçalhos:

```
curl -I http://localhost:8080/
```

[NOTA] Pistas Progressivas (F5) - **Pista 1:** Utilize o decorador `@bp.after_request` em `app/main.py` ou crie uma função no factory de `app/__init__.py`. - **Pista 2:** Injete os 4 cabeçalhos essenciais no objeto de resposta HTTP.

Diretrizes de Implementação

- **Middleware Pós-Requisição Centralizado:** Registre um hook no Flask (`@bp.after_request` ou equivalente no factory) para interceptar globalmente todas as respostas HTTP da aplicação.
- **Cabeçalhos OWASP Obrigatórios:** Injete no dicionário `response.headers` os cabeçalhos defensivos essenciais: `X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , `Content-Security-Policy: default-src 'self'` , `Strict-Transport-Security` e `Referrer-Policy: no-referrer` .
- **Roteiro de Depuração CLI:** Para testar os cabeçalhos diretamente com `curl -I` e validar as regras do OWASP ZAP, consulte `aluno/PLANO_DE_TESTES_DEBUGGING.md` .

Prova de Validação

Execute `curl -I http://localhost:8080/` e confirme a presença dos 4 headers nas respostas. Na Pipeline Dinâmica, o step do OWASP ZAP Baseline passará com zero alertas críticos/altos!

---

4. Quality Gate MEDIUM e Gestão de Exceções

4.1 Como disparar o Gate MEDIUM via `workflow_dispatch`

No GitHub Actions, você pode testar a resiliência do seu projeto com critérios mais rígidos:
1. Acesse a aba **Actions** no seu repositório.
2. Selecione o workflow **DevSecOps CI/CD Pipeline**.
3. Clique no botão **Run workflow** (canto direito).
4. No campo suspenso `Nível de Severidade do Quality Gate` , selecione:
`MEDIUM,HIGH,CRITICAL`
5. Clique em **Run workflow**.

4.2 O que esperar do Gate MEDIUM e por que calibrar

O objetivo deste exercício não é punitivo, mas didático. Em ambientes de produção reais, gates em severidade `MEDIUM` frequentemente identificam vulnerabilidades teóricas em pacotes auxiliares ou dependências transitivas que ainda não possuem patch dos mantenedores de código aberto.
Se o pipeline falhar em severidade MEDIUM devido a um pacote sem correção viável, a postura de um Engenheiro de DevSecOps é avaliar o risco real e aplicar uma **Exceção Formal Documentada**.

4.3 Gestão Justificada de Riscos e Exceções via `.trivyignore`

Para instruir o Trivy a aceitar temporariamente um risco conhecido sem travar a entrega contínua, cria-se o arquivo `.trivyignore` na raiz do repositório:

```
# ------------------------------------------------------------------------
# REGISTRO FORMAL DE EXCEÇÃO DE SEGURANÇA  CLUBE AURORA
# ------------------------------------------------------------------------
# CVE: CVE-202X-XXXX
# Severidade: MEDIUM
```

```
# Componente: libxyz
# Justificativa Técnica: Não há correção disponível upstream. A biblioteca é
# utilizada exclusivamente para parsing interno sem exposição à entrada direta
# do usuário. Mitigado por isolamento em contêiner não-root.
# Responsável: Engenheiro(a) DevSecOps (Seu Nome)
# Data de Expiração / Revisão: 2026-12-31
# -------------------------------------------------------------------------
CVE-202X-XXXX
```

Documente essa análise no seu relatório para demonstrar maturidade de governança de riscos!

---

5. Testes e Validação Contínua

5.1 Testes Unitários e de Fumaça (`pytest -v`)

Suas correções de segurança não devem quebrar as regras de negócio existentes. Execute os testes na raiz do projeto:

```
pytest -v
```

Todos os testes em `tests/test_smoke.py` devem passar com status verde.

5.2 Validação de Cabeçalhos em Tempo de Execução (`curl -I`)

```
curl -I http://localhost:8080/health
```

Certifique-se de que a resposta HTTP contém:

- `HTTP/1.1 200 OK`
- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY`
- `Content-Security-Policy: default-src 'self'`

5.3 Validação Estrutural Local com o Script da Professora

Você pode auditar se as 7 correções atendem aos critérios do gabarito antes de abrir o Pull Request:

```
# Se tiver Python na máquina:
python professor/validate.py .

# Se estiver no Windows PowerShell:
powershell -ExecutionPolicy Bypass -File professor/validate.ps1 .
```

O script testará as 7 falhas e deve relatar `7 de 7 falhas devidamente corrigidas!`

---

6. Desafios Bônus (Além do Escopo Obrigatório)

Estes itens não são obrigatórios para a aprovação com nota máxima, mas enriquecem enormemente seu portfólio profissional:

6.1 Análise e Validação do SBOM (Software Bill of Materials)

Gere o inventário formal de componentes de software (SBOM) no padrão SPDX ou CycloneDX:

```
docker run --rm -v "${PWD}:/src" aquasec/trivy:0.51.2 fs --format spdx-json --output /src/sbom.json /src
```

6.2 Regras de Proteção de Branch (Branch Protection Rules)

Se o seu repositório for público (ou tiver plano GitHub Pro/Enterprise):
- Acesse **Settings Branches Add branch protection rule**.
- Marque **Require a pull request before merging**.
- Marque **Require status checks to pass before merging** e exija que os jobs `pipeline-estatica` e `pipeline-dinamica` estejam verdes para permitir o merge.

6.3 Fixação de Imagem Base por Digest SHA256

Em vez de depender apenas da tag `python:3.11-slim`, você pode fixar a imagem base pelo seu hash imutável SHA256:

```
FROM python:3.11-slim@sha256:c94dc4ef7e9ff83b4b88d3a778c1df95e926ab4509748b6bbdb6abf0a20e2ef5
```

Isso impede ataques de poisoning ou adulteração na cadeia de suprimentos de contêineres.

6.4 Sanitização Profunda de Histórico Git

Ao remover o arquivo `.env` com `git rm --cached`, os segredos ainda permanecem nos commits anteriores do histórico. Em um cenário real de incidente, utilize a ferramenta `git-filter-repo` ou `BFG Repo-Cleaner` para reescrever a árvore do Git e expurgar o arquivo de todos os commits passados.

6.5 Assinatura Digital de Imagens com Cosign

Adicione ao pipeline um step com o utilitário **Cosign** (da Sigstore) para assinar criptograficamente a imagem gerada no GitHub Container Registry (GHCR) via OIDC (OpenID Connect sem chaves estáticas).

---

7. Critérios de Pontuação e Aproveitamento Parcial

7.1 Mapeamento da Rúbrica para Evidências Concretas

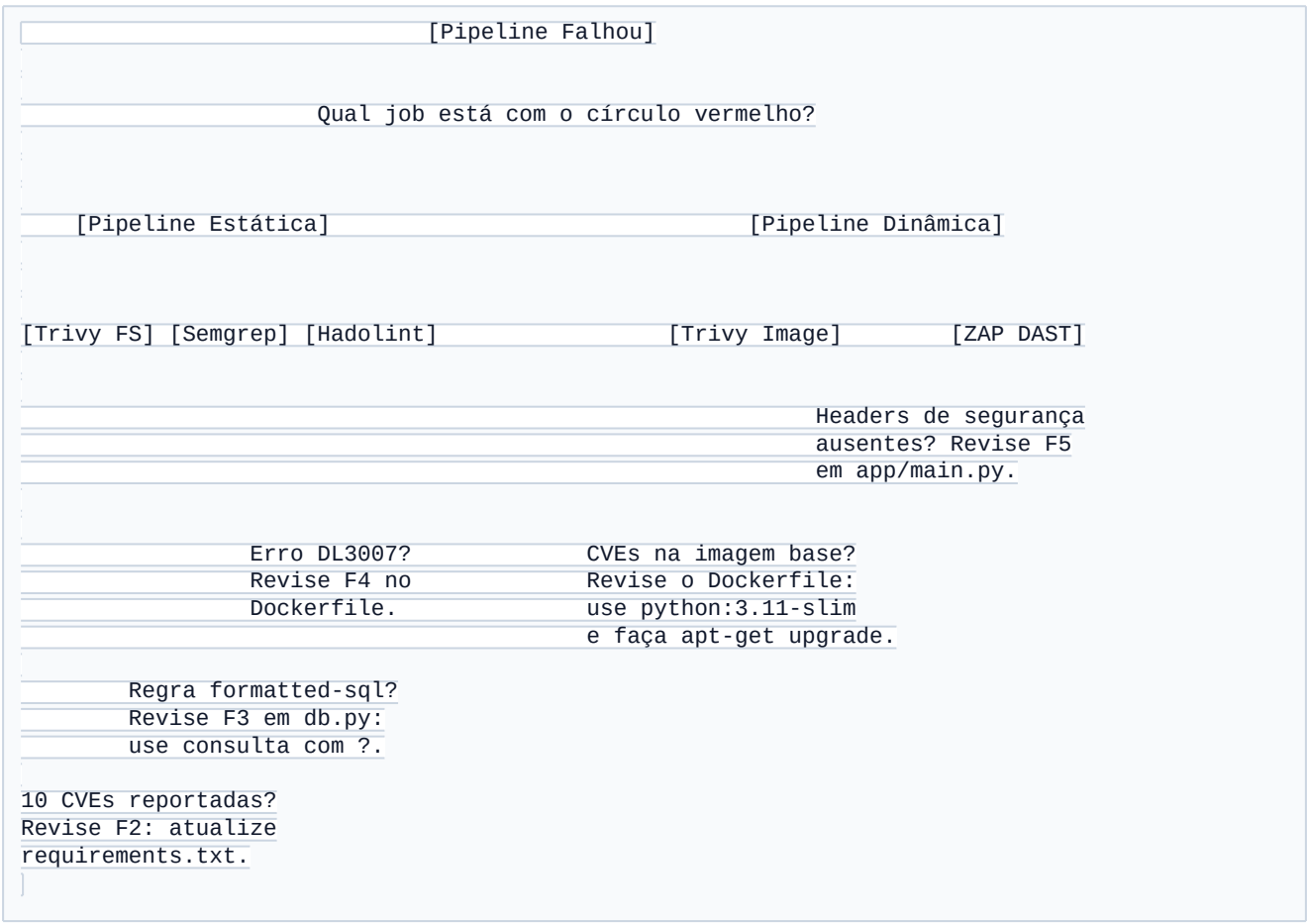
A avaliação técnica é baseada no arquivo `aluno/RUBRICA.md`:
- **Pipeline Verde (Execução Real):** 4,0 pontos.
- **Qualidade das Correções no Código (F1 a F7):** 3,5 pontos.
- **Relatório de Evidências (`TEMPLATE_EVIDENCIAS.md`):** 2,5 pontos.

7.2 Por que entregas parciais têm alto valor

O Nível 3 é um desafio de engenharia real. Se você encontrar dificuldades e conseguir resolver, por exemplo, **5 das 7 falhas** (ex: F1, F2, F3, F4 e F6), mas mantiver o repositório organizado, documentar detalhadamente o que conseguiu e justificar o que faltou, **sua nota ainda será alta e de excelência**.
A transparência e a precisão do seu relatório têm tanto peso quanto o código. Nunca deixe de entregar por não ter atingido os 100%!

---

8. Árvore de Decisão: "Se Você Travar"



Se persistir bloqueado por mais de 30 minutos:

1. Abra o arquivo COMO_LER_OS_RELATORIOS.md;
2. Pergunte no canal da turma detalhando o erro específico;
3. Chame a **professora** com o print do log do GitHub Actions e o comando executado.